Module 04: Cache Coherence

Owner: Prachi Dubey

Team: LNCT Qubit — EPAM CoE: Hardware Concurrency, Memory Models & Lock-Free Design

Date: 2026-10-09

Next handoff: Vivansh — detailed MESI/MOESI state diagrams

Evidence labels: EXTERNAL EVIDENCE, THEORETICAL, STATICALLY REVIEWED, PENDING LOCAL EXECUTION

Status: This is a source-backed conceptual module. It contains no hardware measurements and no claim that cache events were captured on the team's machine. The timelines are abstract event traces for Vivansh to refine into exact protocol-state diagrams.

Scope and boundaries

In scope

- Why cache coherence is required when multiple cores cache the same memory location.
- Cache coherence versus memory consistency / Java Memory Model (JMM) ordering.
- Three event scenarios: read-only sharing, a write after reads, and competing writes.
- Conceptual line ownership, invalidation, cache-to-cache transfer and coherence traffic.
- A neutral event timeline for the next module and a hypothesis for the future false-sharing experiment.

Out of scope

- Repeating the L1/L2/L3 hierarchy, cache capacities, cache-line sizing or locality material in `docs/module03-cpu-cache.md` (Ujjwal).
- Detailed MESI/MOESI state-by-state transitions and exact message/state labels (Vivansh owns these).
- Memory fences, compiler/CPU reordering, CAS internals and lock-free data structures (other modules own these).
- Measuring false sharing or claiming hardware-counter evidence (Vishwajeet's benchmark owns this).

No Java source was added for this module: ordinary Java code cannot directly expose a processor's coherence messages or prove a particular cache-state transition. A Java timing demo would not be evidence of invalidation by itself. This is intentional and matches Task 4's allowance for a Java illustration only when it adds evidence.

The cache-coherence problem

Suppose physical memory location `x` contains `0`. Core C0 reads `x`, so a local cache can retain a copy. Later, core C1 also reads `x` and can retain a copy. If C0 then writes `x = 1`, C1 must not continue treating its old cached copy (`0`) as the current value for a subsequent coherent read.

The coherence mechanism coordinates copies of a cache line so writes to a particular memory location are observed in a coherent order. Coherence is tracked at a cache-line granularity even when the program changes only one field or word. Thus, other bytes in the same line can be involved in the protocol even when they are not logically related to the write.

The required invariant, stated conceptually, is: **cores must not indefinitely use conflicting stale cached copies of the same coherent memory location after a write has been ordered and made visible by the hardware coherence mechanism.** The implementation needs a way to identify which caches may hold a line, grant write permission, and prevent other cached copies from being used as valid copies after a conflicting write.

This does not mean every read must go to DRAM. A core may read a local valid copy; a modified copy may supply data to another core; and a shared last-level cache, coherence directory, snoop filter, or other fabric logic may participate. The path

depends on the processor design.

Cache coherence is not memory consistency

Concept	Question answered	Scope	What it does not guarantee by itself
Cache coherence	For one memory location, how are cached copies and writes coordinated?	A particular location / cache line	It does not define the complete ordering of accesses to different locations.
Memory consistency / memory model	Which ordering of memory operations may a program observe across locations and threads?	Relations among multiple memory actions	It does not prescribe one universal hardware protocol or require every access to travel to main memory.
Java Memory Model	Which inter-thread observations are legal for a Java program?	Java actions, program order, synchronizes-with and happens-before	It does not expose MESI messages or require Java code to implement a particular cache protocol.

Example: a processor can maintain coherent copies of `data` and `ready` individually, but a Java program still needs a valid synchronization relationship if one thread writes `data` and then sets `ready` while another thread checks `ready` before reading `data`. Per-location coherence alone is not a substitute for JMM happens-before rules. For Java, use correctly designed synchronization such as a monitor, a volatile publication flag, or suitable concurrency utilities; the exact choice depends on the program.

Guarantee boundary: JLS §17.4 defines Java's memory model and happens-before relationships. Hardware coherence is a processor-level mechanism. The JLS does not promise that Java variables map one-to-one to cache lines, nor does it specify MESI/MOESI states.

Terminology

- **Cache line:** The hardware's unit of cache storage/coherence in this discussion. Its size is processor-specific; do not assume 64 bytes without checking the tested machine.
- **Coherence:** Coordination that prevents conflicting cached copies of one location from being used as though they were simultaneously current.
- **Read sharing:** Multiple cores hold readable copies of the same line. This is normal and often efficient when no writes occur.
- **Write permission / ownership:** A core obtains the right to modify a line under its coherence protocol. The exact state and request names are protocol-specific.
- **Invalidation:** A coherence action that marks another cached copy as no longer usable as a valid copy for ordinary reads. The other core must reacquire the line before using it again.
- **Snoop-based protocol:** Requests may be observed by other caches through a snooping mechanism. Details depend on the interconnect and implementation.
- **Directory-based protocol:** Metadata tracks where lines may be cached and directs coherence requests. A design may use directories, snoop filters, broadcast snoops, or hybrids.
- **Cache-to-cache transfer:** Data is supplied by another cache or coherence agent rather than fetched only from DRAM. Whether this happens for a specific event depends on the implementation and state.
- **Coherence traffic:** Requests, invalidations, acknowledgements, data responses and related fabric activity needed to coordinate line copies.

Worked event traces

These are **THEORETICAL, protocol-neutral traces**. `C0` and `C1` are logical cores; `x` is a shared variable; `L(x)` is the cache line containing `x`. They describe the logical events that must be explained, not a captured hardware trace. Exact MESI/MOESI state labels and messages are intentionally left to Vivansh.

Scenario 1 — Read-only sharing

Initial condition: `x = 0`; neither core is assumed to have a valid local copy.

Step	Core / coherence event	Conceptual outcome
1	C0 reads `x` and misses locally.	C0 requests the line containing `x` through the platform's coherence path.
2	The coherence path supplies `L(x)` to C0.	C0 can read `x = 0` from its valid local copy.
3	C1 reads `x` and misses locally.	C1 requests the same line. The system coordinates with any existing copy as required by its protocol.
4	C1 receives a valid copy of `L(x)`.	Both cores can read `x = 0` without each read needing to fetch from DRAM.
5	Neither core writes the line.	Read-only sharing generally does not cause repeated write invalidations; copies may remain readable until eviction or a conflicting event.

Point: Sharing a read-only line is not inherently a problem. Multiple clean copies can coexist in protocols designed to allow it. The exact state names and data source are architecture-dependent.

Scenario 2 — One core writes after another core has read

Initial condition: C0 and C1 have both read `x = 0` and may each have a cached copy of `L(x)`.

Step	Core / coherence event	Conceptual outcome
1	C0 requests permission to write `L(x)`.	The coherence system determines which other agents may hold copies and coordinates the permission change.
2	Other cached copies that conflict with the write are invalidated, or equivalent protocol actions are performed.	C1 must not keep using its old copy as a valid current copy after the write is ordered. The writer may need to wait for required acknowledgements.
3	C0 obtains the required write permission and stores `x = 1`.	C0 now owns the line in a state that permits the write under the actual protocol.
4	C1 later reads `x`.	If C1's prior copy was invalidated, its read must reacquire valid data. Data may come from C0's cache or another coherence point; it need not always come from DRAM.

Point: The key cost is not simply “writing a variable”; it is obtaining write permission for the line and coordinating with other possible copies. Do not label the writer's exact state as `M`, `E`, or another state until the protocol-specific diagram is supplied.

Scenario 3 — Competing writes from multiple cores

Initial condition: C0 and C1 repeatedly write the same `x` from different cores.

Step	Event	Conceptual outcome
1	C0 obtains write permission for `L(x)` and writes `x = 1`.	C0 can perform its write while it has the necessary permission.
2	C1 requests write permission for the same line.	C1's request conflicts with C0's current permission; coherence must coordinate a transfer or reacquisition.
3	C0's permission is withdrawn or downgraded as required; C1 obtains permission and writes `x = 2`.	Ownership effectively moves to C1; messages and possibly data transfer are required.
4	C0 writes again.	The line must be coordinated back to C0. Repeated alternation can create repeated coherence traffic and stalls.
5	The two cores continue alternating writes.	The line may “bounce” between cores. The amount of traffic and delay depends on scheduling, topology, protocol, line placement and implementation.

Important correctness caveat: Hardware coherence does not make a compound Java operation such as `count++` atomic. If multiple Java threads perform unsynchronized read-modify-write updates, lost updates can still occur (see Module 01). Coherence coordinates cached copies; Java synchronization/atomic operations provide the language-level semantics required for correct concurrent algorithms.

Abstract event timeline for Vivansh

The following is the handoff skeleton. Replace the conceptual events with exact protocol states, requests, responses and transition conditions from an authoritative manual or protocol specification. Do not present this table as a verified state diagram.

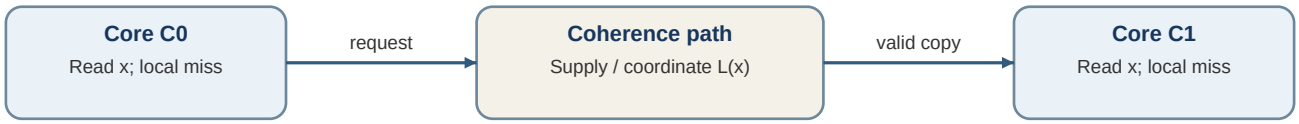
Case	Initial facts	Required event sequence to refine	Question for the exact protocol diagram
Read-only sharing	C0 reads x; C1 then reads x; no writes	local miss → read request → line supplied → second read request → shared valid copies or equivalent outcome	Which states can coexist after both reads? Is the second read served by a cache, shared cache, directory, or memory in this protocol?
Write after read sharing	C0 and C1 both read x; C0 writes	write-permission request → sharer discovery / snoop or directory action → invalidation or downgrade responses → permission granted → write → C1's later read reacquires data	Which transitions invalidate C1? Are acknowledgements required before the writer proceeds? Can a cache-to-cache response supply the new data?
Competing writers	C0 and C1 alternately write x	C0 obtains write permission → C1 requests write permission → C0's permission/copy is changed → C1 writes → C0 requests again	What are the exact states and messages on each transfer? Which agent serializes requests to the same line?

Do not assume a single global “bus” exists. A snooping design, a directory-based design, and a hierarchical coherent fabric can implement the same high-level objective using different traffic and metadata. Even within a vendor family, exact protocol details may vary by processor generation and core cluster.

Supplement: Conceptual coherence transition diagrams

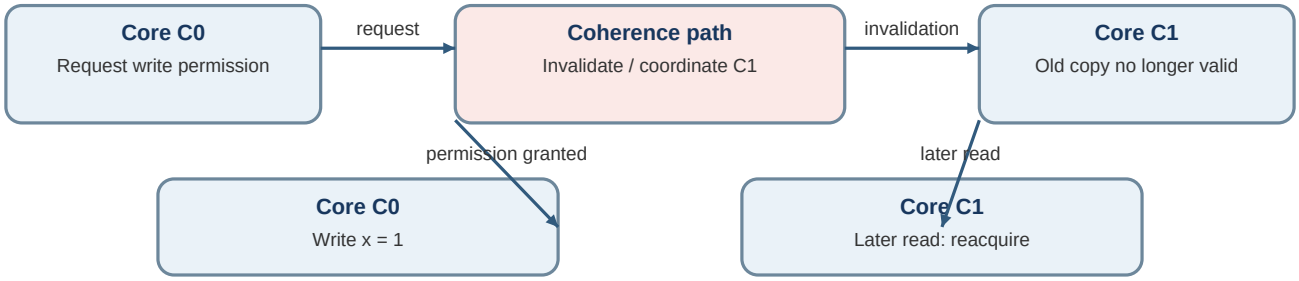
Protocol-neutral event flow — not exact MESI/MOESI state diagrams or captured hardware traces.

1. Read-only sharing

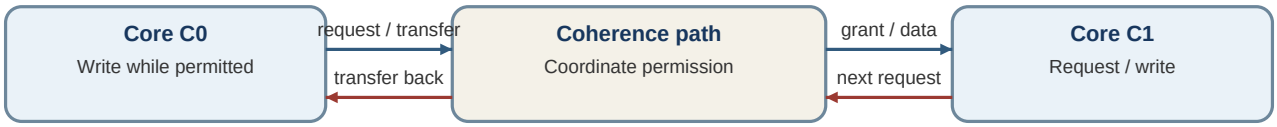


Result: C0 and C1 may both retain readable copies while no write conflicts.

2. C0 writes after both cores read



3. Competing writes (line ownership repeatedly moves)



Repeated alternation can increase coherence traffic and stalls; exact messages depend on the CPU.

Important: these diagrams show logical events only; Vivansh owns the exact MESI/MOESI state labels.

Connection to Vishwajeet's future false-sharing benchmark

Hypothesis only — not a result: if two threads repeatedly write different variables that happen to occupy the same coherence line, throughput may be lower than when the variables are placed on separate lines. In the shared-line case, each writer's write permission may conflict with the other writer's cached copy of that line, causing invalidation/ownership traffic even though the source-level variables are independent. This is the classic false-sharing mechanism.

Experiment item	Proposed definition for Vishwajeet
Independent variable	Layout: two independently updated counters on the same cache line versus counters separated/aligned so they occupy different lines.
Dependent variables	Throughput (operations/second) and/or elapsed time for a fixed operation count; report spread across forks/runs.
Controls	Same CPU/JDK, thread count and placement policy where possible, update operation, iteration count, warm-up, measurement time, and benchmark harness. Ensure the counters are independently owned by one writer each.
Required checks	Confirm layout/padding assumptions for the JVM and object layout; do not assume Java object fields or padding land on a specific line. Record CPU model, core type/affinity if available, JDK, OS, JMH version and command line.
Stronger evidence	If available, collect compatible hardware performance counters or use a validated profiler to inspect cache-to-cache/HITM or relevant coherence events for the exact CPU. Counter availability and event meanings must be checked against vendor documentation.
Interpretation	A timing difference is evidence of a performance difference under that setup, not by itself proof of MESI transitions, cache misses or invalidation count. No difference also does not disprove false sharing; compiler/JVM layout, scheduling, CPU placement and measurement noise can mask it.
Status	**PENDING** — no false-sharing benchmark was run as part of this module.

The experiment should include at least one correctness check that both counters reach their expected final values, but its performance measurement must avoid introducing a shared synchronization bottleneck into the measured update loop unless that is deliberately part of the test.

Source-backed claims and references

Claim used here	Authoritative source	Exact location	Scope / limitation
Intel documents MESI as a cache protocol for coordinating cached copies across processors; the states encode validity, exclusivity/sharing and modification status.	Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Volume 3A: System Programming Guide, Part 1. Official PDF	§11.2 "Caching Terminology" and §11.4 "Cache Control Protocol", especially Table 11-4, printed pp. 11-6 to 11-10 in this edition.	Intel 64/IA-32 documentation, not a universal specification for all CPU families; use the current manual for the exact host generation before claiming exact implementation details.

Claim used here	Authoritative source	Exact location	Scope / limitation
Hardware cache coherence can migrate lines between cores, while memory ordering is a separate concern; transactions may be buffered/reordered.	Arm, *Learn the architecture — ARMv8-A memory systems*. Official documentation	“The memory model” and “Cache coherent cluster processing”.	Arm architecture educational documentation; illustrates that coherency and ordering are related but distinct. It is not an exact protocol-state manual for the team's host CPU.
A coherent network can track cache-line locations and use snoop filters/direct snoops rather than requiring every request to be broadcast to every core.	Arm, *CoreLink CCN-508 Cache Coherent Network Technical Reference Manual*. Official documentation	“Fully-coherent Home Node”; description of the home node, point of serialization/coherency and snoop filter.	Example of one Arm interconnect implementation; not a claim that Intel or all Arm CPUs use this exact design.
Java's legal inter-thread observations are defined through program order, synchronization order and happens-before; data races are defined in the JMM.	Oracle, *The Java Language Specification, Java SE 21 Edition*. Chapter 17	§§17.4.1–17.4.5, especially §17.4.5 “Happens-before Order”.	Java-language guarantee, independent of a particular physical cache-coherence protocol. Use the edition matching the project's selected Java baseline when possible.

Interpretation and limitations

1. The traces are conceptual and are not captured snoop logs, simulator output or physical hardware traces.
2. No local Java code can, by itself, expose the exact coherence messages generated by a particular CPU. No Java timing result alone proves cache-line bouncing, cache misses, MESI transitions or physical invalidations.
3. Intel's MESI description is useful for an Intel-specific example, but other processors may use different state machines, MOESI-family variants, directories, snoop filters or proprietary/hierarchical protocols. Vivansh must state the target architecture and whether each diagram is generic or host-specific.
4. The project documents a hybrid Intel Core i7-13700HX in prior-module context, but this module has not independently collected the exact machine's current topology, cache line size, core placement or hardware-counter output. Those values remain to be recorded from the machine used for any benchmark.
5. A write to one field can trigger coherence for the whole line. This is why unrelated fields can interact through false sharing, but layout must be verified rather than assumed from Java source declarations.
6. Cache coherence is not a replacement for JMM synchronization, atomic read-modify-write operations, or correct publication.

HANDOFF PACKET · Prachi · Vivansh

Project objective and shared decisions

Build one integrated, evidence-based engineering project on hardware concurrency, memory models and lock-free design. Preserve the existing Maven project, package-per-module convention, `docs/` location, `results/raw/` data convention, and evidence labels. Use only real measurements; label unexecuted work as pending. Module 03 (`docs/module03-cpu-cache.md`) owns hierarchy, locality and single-threaded JMH baseline work.

Files added / changed

- Added: `docs/module04-cache-coherence.md` — this module, its three event traces, source references, limits and handoff.
- Updated: `docs/references.md` — append the Module 04 source claims.
- No Java source, Maven dependency, test or raw result file added/changed. The module does not claim a hardware experiment was run.

Public classes, methods and interfaces

None. This module is conceptual documentation; no API or source-code contract is introduced.

Established facts and sources to carry forward

- Coherence coordinates copies and writes to a particular memory location/cache line.
- Memory consistency/JMM describes legal ordering and observations across memory actions; it is not interchangeable with cache coherence.
- Read-only sharing can permit multiple valid copies; a writer must coordinate conflicting copies; competing writers can repeatedly transfer write permission.
- Intel SDM Vol. 3A §11.4 describes MESI for Intel 64/IA-32; Arm's coherent-network documentation illustrates directory/snoop-filter style mechanisms. Neither source justifies saying every CPU uses the same protocol.
- JLS SE 21 Chapter 17 is the Java-language source for happens-before and data-race definitions.

Event sequences requiring exact MESI/MOESI diagrams

1. **Read-only sharing:** C0 read miss and line fill; C1 read of the same line; show whether/how both copies can be valid and the state after each request.
2. **Write after sharing:** C0 and C1 have read the line; C0 requests write permission; show snoop/directory action, invalidation/downgrade responses, grant, store, and C1's later read/data source.
3. **Competing writes:** C0 owns write permission and stores; C1 requests permission and stores; C0 requests it again. Show exact states/messages and the serialization point for the same line.

For every diagram, name the architecture/protocol source, state whether it is a generic teaching model or exact host-specific design, and do not invent proprietary messages.

Dependencies and integration requirements

- Documentation only; no Maven dependency or API integration needed.
- Preserve `docs/module03-cpu-cache.md`; do not duplicate its hierarchy diagram.
- Keep the conceptual event traces aligned with the state names and transition rules that Vivansh verifies from the selected protocol source.
- Coordinate with Vishwajeet on the same-line versus separated-line false-sharing hypothesis; do not report a benchmark outcome before execution.

Tests actually run / status

- No Java build, unit test, JMH benchmark, hardware-counter capture or cache-coherence experiment was run for this module.
- Source links and relevant sections were checked during document preparation; re-check them against the exact manual version chosen for final review.
- Status: **STATICALLY REVIEWED / PENDING LOCAL REVIEW**. No TEAM-MEASURED claims are made.

Open issues / assumptions

- Formal Project Contract was not present as a separate artifact in the supplied project snapshot; current conventions are inferred from the integrated repo and master instructions.
- Confirm the target CPU and whether Vivansh will draw generic MESI/MOESI or the exact host implementation.
- Confirm current cache-line size, topology and available performance-counter events on the actual benchmark machine before interpreting Vishwajeet's results.
- The task owner change from the older PDF is acknowledged: Prachi owns Task 4 in the current assignment and hands off to Vivansh; the old PDF's Kabir ownership is superseded.

Exact inputs the next member needs

- This file, especially Sections E, F and H.
- The actual target architecture/CPU model and the exact vendor manual edition to use.
- A decision on the named protocol and the intended diagram scope (generic teaching protocol versus host-specific implementation).
- Any review feedback from the mentor that requests a particular protocol, state transition, or architecture diagram.

Final self-audit

Requirement	Status	Evidence / next action
Explain the coherence problem	**Covered conceptually**	Sections B and E.
Distinguish coherence from consistency/JMM	**Covered**	Section C, sourced to JLS Chapter 17.
Three required scenarios	**Covered as theoretical traces**	Section E.
Ownership, invalidation and coherence traffic	**Covered conceptually**	Sections D and E.
Timeline for Vivansh	**Provided; protocol states pending**	Section F and handoff.
False-sharing hypothesis	**Provided; outcome pending**	Section G.
Authoritative architecture references	**Provided**	Section H; confirm the chosen manual edition during final review.
Avoid claims that all CPUs use the same protocol	**Covered**	Sections F and I.
Java code / measurements	**Intentionally omitted**	Code cannot expose coherence messages; no experiment was run.
Handoff packet	**Provided**	Section L.
Head of EPAM India's feedback	**Not available in supplied documents**	This audit checks the master instructions and Task 4 brief only; re-audit when feedback is provided.